

```
"""
Shared option strategy: 3m setup + 1m trigger + post-entry monitor.
Used by NSE-Option-Scanner.py (live) and NSE-Option-Scanner-Backtest.py.
"""
```

```
import datetime
```

```
import pandas as pd
```

```
# --- scoring weights ---
STRONG_BUY_THRESHOLD = 90
TARGET_PERCENT = 0.20
SL_PERCENT = 0.10
TRAIL_PERCENT = 0.05
PROFIT_TRAIL_START = 0.05
```

```
SETUP_TIMEFRAME = 3 # macro trend (was 5m)
TRIGGER_TIMEFRAME = 1
SETUP_MIN_BARS = 25
```

```
# --- Order Book Imbalance (OBI) ---
# Only take a 1m CE trigger if OBI is positive and strong (buyer
dominance).
MIN_ORDER_BOOK_IMBALANCE_CE = 0.2
```

```
EMA_FAST = 9
EMA_SLOW = 20
RSI_PERIOD = 14
SUPERTREND_PERIOD = 10
SUPERTREND_MULTIPLIER = 3
```

```
WEIGHT_TREND = 25
WEIGHT_OI = 25
WEIGHT_VOLUME = 20
WEIGHT_IV = 15
WEIGHT_ATM = 15
MAX_SCORE = WEIGHT_TREND + WEIGHT_OI + WEIGHT_VOLUME + WEIGHT_IV +
WEIGHT_ATM
```

```
IST = "Asia/Kolkata"
MARKET_EXIT_TIME = datetime.time(15, 10)
MARKET_ENTRY_CUTOFF = datetime.time(14, 30) # backtest: late entries
underperform
```

```
# Live scanner intervals (seconds)
SCAN_INTERVAL_IDLE = 5
SCAN_INTERVAL_SIDEWAYS = 8
SCAN_SETUP_REFRESH = 20
SCAN_CHAIN_REFRESH = 8
MONITOR_INTERVAL = 2
MONITOR_FULL_REFRESH = 15
SCAN_HEARTBEAT = 30
```

```
def to_ist(ts):
    """Convert API timestamp to IST (API returns UTC)."""
    dt = pd.to_datetime(ts)
    if dt.tzinfo is None:
```

```

        dt = dt.tz_localize("UTC")
        return dt.tz_convert(IST)

def format_ist(ts):
    return to_ist(ts).strftime("%Y-%m-%d %H:%M:%S IST")

def format_ist_time(ts):
    return to_ist(ts).strftime("%H:%M IST")

def calculate_rsi_series(df, period=RSI_PERIOD):
    delta = df["close"].diff()
    gain = delta.where(delta > 0, 0).rolling(period).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(period).mean()
    rs = gain / loss.replace(0, pd.NA)
    return 100 - (100 / (1 + rs))

def calculate_rsi(df, period=RSI_PERIOD):
    series = calculate_rsi_series(df, period)
    return series.iloc[-2] if len(series) >= 2 else series.iloc[-1]

def add_supertrend(df, period=SUPERTREND_PERIOD,
multiplier=SUPERTREND_MULTIPLIER):
    """TradingView-style Supertrend; adds supertrend + st_color
    (GREEN/RED)."""
    work = df.copy()
    if work.empty:
        work["supertrend"] = pd.Series(dtype=float)
        work["st_color"] = pd.Series(dtype=str)
        return work

    high = work["high"].astype(float)
    low = work["low"].astype(float)
    close = work["close"].astype(float)

    tr = pd.concat(
        [high - low, (high - close.shift()).abs(), (low -
close.shift()).abs()],
        axis=1,
    ).max(axis=1)
    atr = tr.rolling(period).mean()

    hl2 = (high + low) / 2
    upper = hl2 + multiplier * atr
    lower = hl2 - multiplier * atr

    st = [float(lower.iloc[0])]
    direction = [1]
    ub = upper.tolist()
    lb = lower.tolist()
    closes = close.tolist()

    for i in range(1, len(work)):
        if closes[i] > ub[i - 1]:

```

```

        direction.append(1)
    elif closes[i] < lb[i - 1]:
        direction.append(-1)
    else:
        direction.append(direction[-1])
        if direction[-1] == 1 and lb[i] < st[-1]:
            lb[i] = st[-1]
        elif direction[-1] == -1 and ub[i] > st[-1]:
            ub[i] = st[-1]

    st.append(lb[i] if direction[-1] == 1 else ub[i])

work["supertrend"] = st
work["st_color"] = ["GREEN" if d == 1 else "RED" for d in direction]
return work

def st_color_at_row(df, idx=-2):
    if df is None or df.empty:
        return ""
    row_idx = idx if idx >= 0 else len(df) + idx
    if row_idx < 0 or row_idx >= len(df):
        row_idx = len(df) - 1
    if "st_color" not in df.columns:
        df = add_supertrend(df)
    return str(df.iloc[row_idx]["st_color"])

def add_vwap(df):
    work = df.copy()
    typical = (work["high"] + work["low"] + work["close"]) / 3
    vol = work["volume"].replace(0, pd.NA)
    work["vwap"] = (typical * work["volume"]).cumsum() / vol.cumsum()
    return work

def last_completed_bar_time(df):
    """Timestamp of last fully closed candle (iloc[-2])."""
    if df is None or df.empty or len(df) < 2:
        return None
    return pd.to_datetime(df.iloc[-2]["start_Time"])

def is_entry_time_allowed(bar_time=None):
    """Backtest: skip entries after 14:30 IST â€” late-day chop loses more."""
    if bar_time is None:
        t = datetime.datetime.now().time()
    else:
        t = to_ist(bar_time).time()
    return t < MARKET_ENTRY_CUTOFF

def prepare_nifty_frames(df_setup, df_1m):
    """Add EMA, RSI, Supertrend to NIFTY setup + 1m frames."""
    work_setup = add_supertrend(df_setup.copy())
    work_setup["EMA9"] = work_setup["close"].ewm(span=EMA_FAST).mean()
    work_setup["EMA20"] = work_setup["close"].ewm(span=EMA_SLOW).mean()

```

```

work_setup["RSI"] = calculate_rsi_series(work_setup)

work_1m = add_supertrend(df_1m.copy())
work_1m["EMA9"] = work_1m["close"].ewm(span=EMA_FAST).mean()
work_1m["EMA20"] = work_1m["close"].ewm(span=EMA_SLOW).mean()
work_1m["RSI"] = calculate_rsi_series(work_1m)
work_1m = add_vwap(work_1m)
return work_setup, work_1m

def build_nifty_indicator_snapshot(df_setup, df_1m, bar_time=None):
    """RSI + Supertrend for monitor logs / CSV / Excel (3m setup +
    1m)."""
    work_setup, work_1m = prepare_nifty_frames(df_setup, df_1m)
    row_setup = work_setup.iloc[-2] if len(work_setup) >= 2 else
work_setup.iloc[-1]
    row_1m = work_1m.iloc[-2] if len(work_1m) >= 2 else work_1m.iloc[-1]
    setup = detect_setup_3m(df_setup)

    return {
        "nifty_close": float(row_1m["close"]),
        "vwap": float(row_1m["vwap"]),
        "ema9": float(row_setup["EMA9"]),
        "ema20": float(row_setup["EMA20"]),
        "rsi": float(row_setup["RSI"]),
        "rsi_3m": round(float(row_setup["RSI"]), 2),
        "rsi_1m": round(float(row_1m["RSI"]), 2),
        "supertrend_3m": str(row_setup["st_color"]),
        "supertrend_1m": str(row_1m["st_color"]),
        "nifty_trend": setup["trend"] if setup else "SIDEWAYS",
        "bar_time": bar_time or row_1m.get("start_Time",
datetime.datetime.now()),
    }

def detect_setup_3m(df_3m):
    """
    3-min macro setup: EMA + RSI + Supertrend on completed candle.
    """
    if df_3m is None or df_3m.empty or len(df_3m) < SETUP_MIN_BARS:
        return None
    work, _ = prepare_nifty_frames(df_3m, df_3m)
    row = work.iloc[-2]
    trend = "SIDEWAYS"
    st = str(row["st_color"])
    if row["EMA9"] > row["EMA20"] and row["RSI"] > 55 and st == "GREEN":
        trend = "BULLISH"
    elif row["EMA9"] < row["EMA20"] and row["RSI"] < 45 and st == "RED":
        trend = "BEARISH"
    return {
        "trend": trend,
        "ema9": float(row["EMA9"]),
        "ema20": float(row["EMA20"]),
        "rsi": float(row["RSI"]),
        "supertrend": st,
        "close": float(row["close"]),
    }

```

```

def detect_setup_5m(df_5m):
    """Backward-compatible alias â€” use 3m setup."""
    return detect_setup_3m(df_5m)

def calculate_order_book_imbalance(market_depth):
    """
    OBI = (TotalBidQty - TotalAskQty) / (TotalBidQty + TotalAskQty)

    Accepts either:
    - dict with totals: total_buy_quantity / total_sell_quantity
    - dict with lists: bids=[{quantity:...}], asks=[{quantity:...}]
    - XTS-like dicts (various casing); best-effort parsing
    """
    if not market_depth:
        return None

    def _first(*vals):
        for v in vals:
            if v is not None:
                return v
        return None

    total_bid_qty = _first(
        market_depth.get("total_buy_quantity"),
        market_depth.get("totalBuyQuantity"),
        market_depth.get("TotalBuyQuantity"),
        market_depth.get("total_bid_quantity"),
        market_depth.get("bid_qty_total"),
    )
    total_ask_qty = _first(
        market_depth.get("total_sell_quantity"),
        market_depth.get("totalSellQuantity"),
        market_depth.get("TotalSellQuantity"),
        market_depth.get("total_ask_quantity"),
        market_depth.get("ask_qty_total"),
    )

    if total_bid_qty is None:
        bids = market_depth.get("bids") or market_depth.get("Bid") or []
        total_bid_qty = sum(int((b or {}).get("quantity", (b or
{})).get("Quantity", 0)) or 0) for b in bids)
        if total_ask_qty is None:
            asks = market_depth.get("asks") or market_depth.get("Ask") or []
            total_ask_qty = sum(int((a or {}).get("quantity", (a or
{})).get("Quantity", 0)) or 0) for a in asks)

    try:
        total_bid_qty = float(total_bid_qty or 0)
        total_ask_qty = float(total_ask_qty or 0)
    except Exception:
        return None

    denom = total_bid_qty + total_ask_qty
    if denom <= 0:
        return None
    return round((total_bid_qty - total_ask_qty) / denom, 3)

```

```

def detect_trigger_1m(df_1m, option_type, market_depth=None):
    """
    1-min trigger: Supertrend color on completed 1m candle.
    CE: GREEN | PE: RED
    """
    if df_1m is None or df_1m.empty or len(df_1m) < 20:
        return False

    # OBI gate: CE only (as per strategy blueprint)
    if option_type == "CE" and market_depth:
        obi = calculate_order_book_imbalance(market_depth)
        if obi is None or obi < MIN_ORDER_BOOK_IMBALANCE_CE:
            return False

    work = add_supertrend(df_1m.copy())
    row = work.iloc[-2]
    st = str(row["st_color"])
    if option_type == "CE":
        return st == "GREEN"
    if option_type == "PE":
        return st == "RED"
    return False


def score_trend(option_type, trend):
    if trend == "BULLISH" and option_type == "CE":
        return WEIGHT_TREND
    if trend == "BEARISH" and option_type == "PE":
        return WEIGHT_TREND
    if trend == "SIDEWAYS":
        return WEIGHT_TREND * 0.4
    return 0


def score_oi(oi_change, max_oi_change):
    if oi_change <= 0 or max_oi_change <= 0:
        return 0
    return (oi_change / max_oi_change) * WEIGHT_OI


def score_volume(volume, max_volume):
    if volume <= 0 or max_volume <= 0:
        return 0
    return (volume / max_volume) * WEIGHT_VOLUME


def score_iv(iv):
    if 12 <= iv <= 20:
        return WEIGHT_IV
    if 10 <= iv <= 25:
        return WEIGHT_IV * 0.55
    if iv > 35:
        return -10
    return 0

```

```

def score_atm(strike, atm, step=50):
    distance = abs(strike - atm) / step
    return max(0, WEIGHT_ATM - distance)

def build_scored_options(chain_data, trend):
    atm = chain_data["atm"]
    options = chain_data["options"]
    if not options:
        return []
    max_oi_change = max(max(o["oi_change"], 0) for o in options) or 1
    max_volume = max(o["volume"] for o in options) or 1
    scored = []
    for opt in options:
        raw = (
            score_trend(opt["option_type"], trend)
            + score_oi(opt["oi_change"], max_oi_change)
            + score_volume(opt["volume"], max_volume)
            + score_iv(opt["iv"])
            + score_atm(opt["strike"], atm)
        )
        score_pct = round(max(0, min(100, (raw / MAX_SCORE) * 100)), 2)
        signal = "STRONG BUY" if score_pct >= STRONG_BUY_THRESHOLD else
"WATCH"
        scored.append(**opt, "score_pct": score_pct, "signal": signal)
    return scored

def pick_best_by_type(scored_options):
    best_ce = best_pe = None
    for opt in scored_options:
        if opt["option_type"] == "CE":
            if best_ce is None or opt["score_pct"] >
best_ce["score_pct"]:
                best_ce = opt
            elif opt["option_type"] == "PE":
                if best_pe is None or opt["score_pct"] >
best_pe["score_pct"]:
                    best_pe = opt
    return best_ce, best_pe

def entry_levels(entry_ltp):
    return {
        "target": round(entry_ltp * (1 + TARGET_PERCENT), 2),
        "sl": round(entry_ltp * (1 - SL_PERCENT), 2),
    }

def format_buy_alert(pick, entry_ltp):
    label = (pick.get("display_symbol") or pick["symbol"]).upper()
    label = label.replace(" CALL", " CE").replace(" PUT", " PE")
    levels = entry_levels(entry_ltp)
    return f"BUY: {label} @ Rs {entry_ltp} | Trailing SL = Rs
{levels['sl']}"

def format_strong_buy_alert(pick):

```

```

label = (pick.get("display_symbol") or pick["symbol"]).upper()
label = label.replace(" CALL", " CE").replace(" PUT", " PE")
return f"{label} -- STRONG BUY"

def _maybe_auto_trail(position, ltp, sl):
    """Ratchet SL up when trade is in profit (backtest: trailing SL beats
    fixed target)."""
    entry = position["entry_ltp"]
    max_ltp = position.get("max_ltp", ltp)
    if ltp < entry * (1 + PROFIT_TRAIL_START):
        return None
    new_sl = round(max_ltp * (1 - TRAIL_PERCENT), 2)
    if new_sl > sl:
        return new_sl
    return None

def monitor_position(position, snapshot):
    """
    Post-entry monitor (priority order).
    Backtest-tuned: trailing SL primary exit; ST flip hard exit;
    EMA cross only when losing.
    """
    side = position["option_type"]
    ltp = snapshot["ltp"]
    sl = position["sl"]
    entry = position["entry_ltp"]

    # Auto-trail when in profit before SL check
    auto_sl = _maybe_auto_trail(position, ltp, sl)
    if auto_sl:
        sl = auto_sl

    if ltp <= sl:
        reason = "STOP_LOSS" if sl <= position.get("initial_sl", sl) else
"TRAILING_SL_HIT"
        return "EXIT", reason, sl

    bar_time = snapshot.get("bar_time")
    if bar_time is not None:
        t = to_ist(bar_time).time()
        if t >= MARKET_EXIT_TIME:
            return "EXIT", "TIME_DECAY", sl

    st_3m = snapshot.get("supertrend_3m", "")
    if side == "CE" and st_3m == "RED":
        return "EXIT", "SUPERTREND_FLIP", sl
    if side == "PE" and st_3m == "GREEN":
        return "EXIT", "SUPERTREND_FLIP", sl

    trend = snapshot.get("nifty_trend", "SIDEWAYS")
    if side == "CE" and trend == "BEARISH":
        return "EXIT", "NIFTY_TREND_REVERSAL", sl
    if side == "PE" and trend == "BULLISH":
        return "EXIT", "NIFTY_TREND_REVERSAL", sl

    reasons = []

```



```

oi_chg = snapshot.get("oi_change", 0)
if side == "CE" and oi_chg < 0:
    reasons.append("OI falling on CE")
if side == "PE" and oi_chg < 0:
    reasons.append("OI falling on PE")

vol = snapshot.get("volume", 0)
prev_vol = snapshot.get("prev_volume", vol)
if prev_vol > 0 and vol < prev_vol * 0.7:
    reasons.append("Volume slowing")

nifty_close = snapshot.get("nifty_close")
vwap = snapshot.get("vwap")
if nifty_close and vwap:
    if side == "CE" and nifty_close < vwap:
        reasons.append("NIFTY below VWAP")
    if side == "PE" and nifty_close > vwap:
        reasons.append("NIFTY above VWAP")

ema9 = snapshot.get("ema9")
ema20 = snapshot.get("ema20")
if ema9 is not None and ema20 is not None and ltp < entry:
    if side == "CE" and ema9 < ema20:
        return "EXIT", "EMA9_CROSSED_BELOW_EMA20", sl
    if side == "PE" and ema9 > ema20:
        return "EXIT", "EMA9_CROSSED_ABOVE_EMA20", sl

rsi = snapshot.get("rsi")
if rsi is not None and ltp < entry:
    if side == "CE" and rsi < 40:
        return "EXIT", "RSI_REVERSAL", sl
    if side == "PE" and rsi > 60:
        return "EXIT", "RSI_REVERSAL", sl

if auto_sl:
    return "TRAIL_STOP", "Profit protection", auto_sl

if reasons and ltp > entry:
    new_sl = round(ltp * (1 - TRAIL_PERCENT), 2)
    if new_sl > sl:
        return "TRAIL_STOP", "; ".join(reasons), new_sl

if reasons:
    return "HOLD", "; ".join(reasons), sl

return "HOLD", "Momentum intact", sl

def classify_outcome(entry_ltp, exit_ltp, exit_reason):
    pnl_pct = round(((exit_ltp - entry_ltp) / entry_ltp) * 100, 2)
    if exit_reason in ("TARGET_HIT", "TRAILING_SL_HIT") and pnl_pct >= 0:
        return "SUCCESS", pnl_pct
    if exit_reason == "STOP_LOSS":
        if pnl_pct >= 0:
            return "SUCCESS", pnl_pct
        return "FAILURE", pnl_pct
    if pnl_pct > 0:

```

```
        return "SUCCESS", pnl_pct
if pnl_pct < 0:
    return "FAILURE", pnl_pct
return "BREAKEVEN", pnl_pct
```